

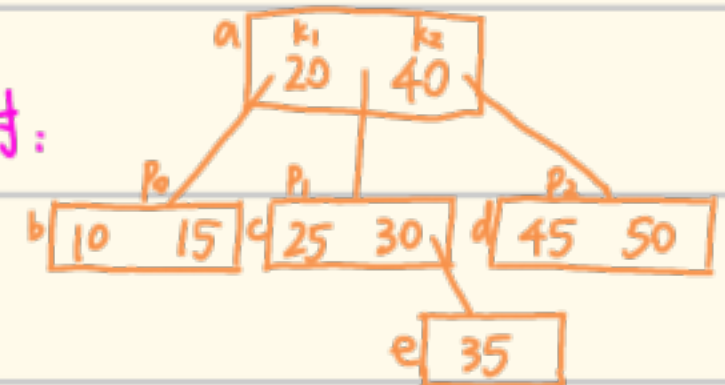
m路搜索树

定义：一棵 m 路搜索树，或者是一棵空树，或者是满足如下性质的树：

- 根最多有 m 棵子树，且有以下结构：(n, P₀, K₁, P₁, K₂, P₂, ..., K_n, P_n)

其中 P_i 指向第 i 棵子树；K_i 为第 i 个关键字，K_i < K_{i+1} (0 ≤ i ≤ n < m)

eg: 如下树即为一棵 3 路搜索树：



- AVL 树即为一棵 2 路搜索树；m 路搜索树若高度为 h，则至多有 $\sum_{i=1}^h m^i = \frac{m^{h+1}-1}{m-1}$

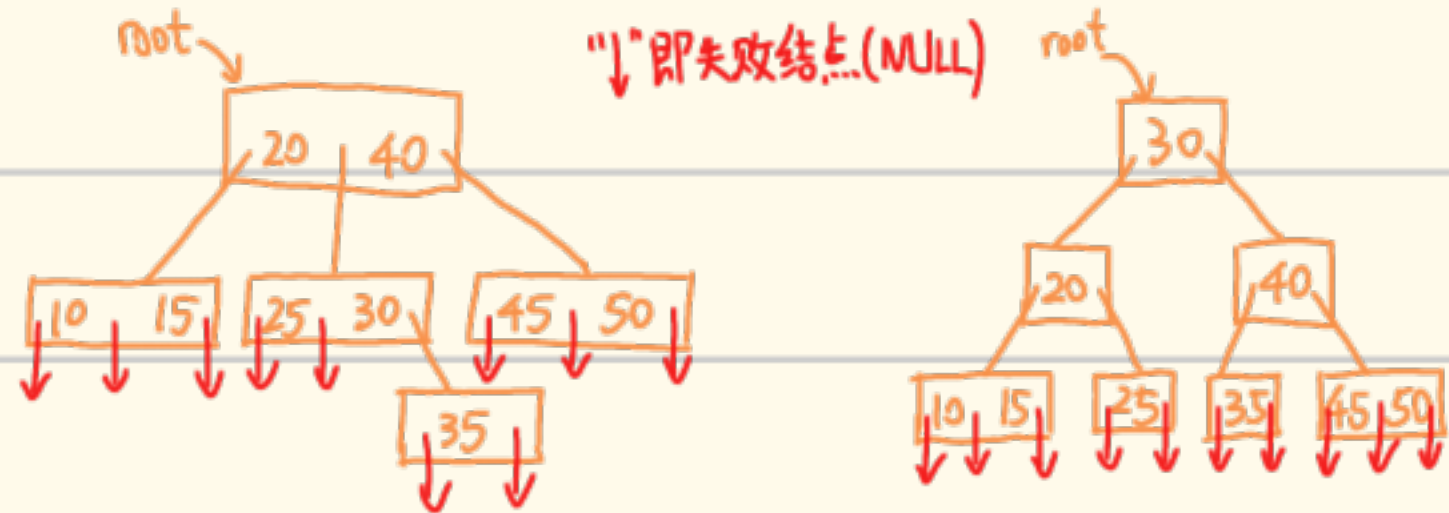
个结点，又因每个结点至多有 m-1 个关键字，故其总共至多有 m^h-1 个关键字

B 树

m 阶 B 树即为一棵较特殊的 m 阶搜索树，其满足以下性质：

- ① 根节点至少有 2 个子女
- ② 除根结点外的所有结点至少有 $\lceil m/2 \rceil$ 个子女
- ③ 所有失败结点（即：当关键字 x 不在树中才能到达的结点，为 NULL）在同一层 (h+1 层)

eg: 3 路搜索树 (非 B 树) 与 3 阶 B 树对比：



B 树的搜索：基本与二叉搜索树相同，搜索时间同时与 m 和树高 h 有关

(搜索成功时间取决于关键字所在的层次；搜索失败时间取决于树高 h)

- 树高 h 范围的推导：(设关键字个数 N, m 阶)

$$\lceil \log_m(N+1) \rceil \leq h \leq \log_{\lceil m/2 \rceil} \frac{N+1}{2} + 1$$

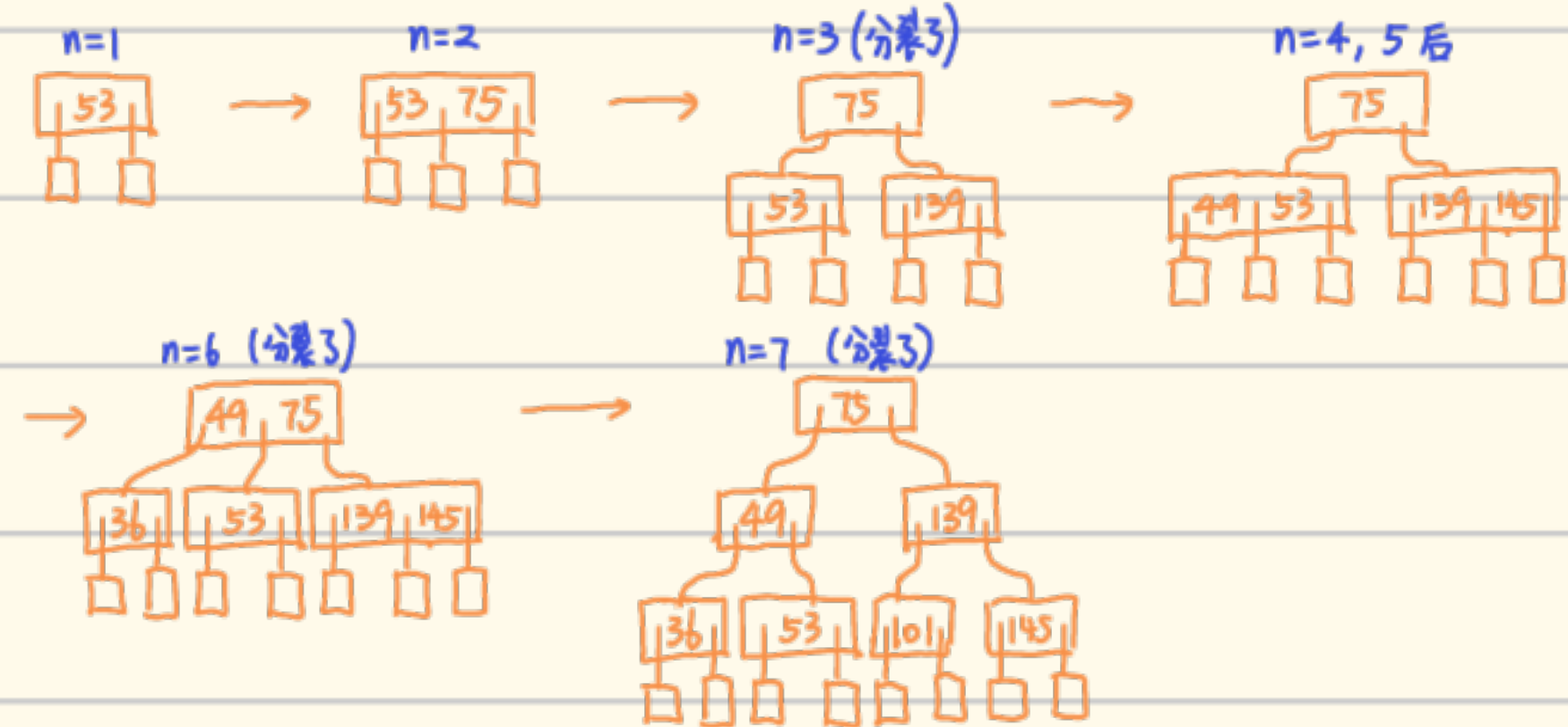
(因为失败结点数为 N+1，均位于第 h+1 层，该层至少有 $2 \lceil m/2 \rceil^{h-1}$ 个结点。)

B 树其他特性：根结点的子树数 ∈ [2, m]，关键字数 ∈ [1, m-1]

其他非失败叶子树数 ∈ [⌈m/2⌉, m]，关键字数 ∈ [⌈m/2⌉-1, m-1]

- B 树的插入：插入在某叶结点处开始，若插入后关键字个数超出了上界 m-1，则对应的结点需进行“分裂”（取正中间的 (K_{⌈m/2⌉}) 作为新根）。否则直接插入。(不可借用兄弟)}

eg: 按 53, 75, 139, 49, 45, 36, 101 的插入顺序建立一个 3 阶 B 树的全过程：

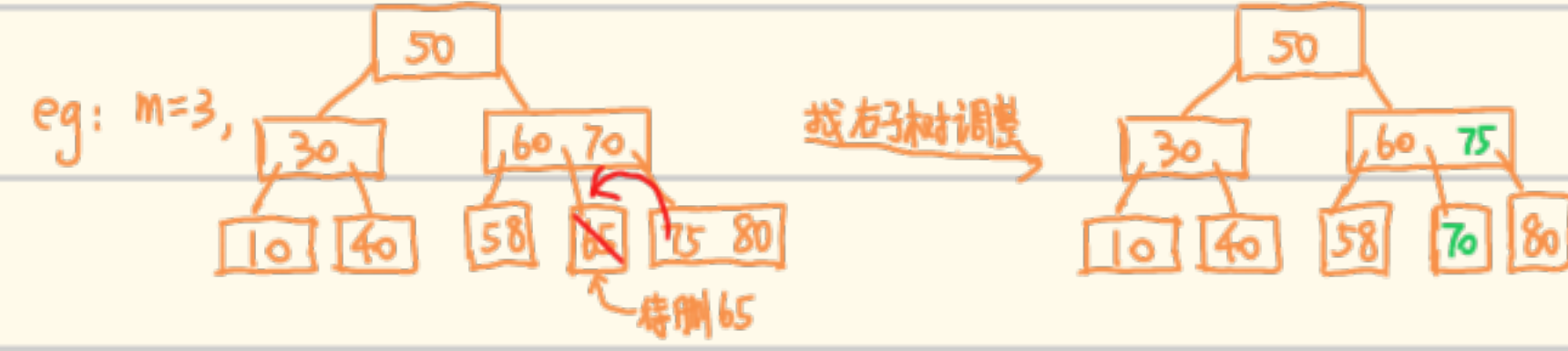


- B 树的删除：先找到待删结点，若不是叶结点，则找该结点右子树的最小关键字代替该结点，然后考虑该叶结点的删除问题，总之删除时只处理叶结点：

- ① 简单删除：删前该结点关键字个数 n ≥ ⌈m/2⌉，则直接删



- ② 调整：若其左/右兄弟有可借用的，则与父结点一起调整即可



- ③ 合并：若其左右兄弟均不可借用，则合并当前删除元素的父结点和兄弟结点。



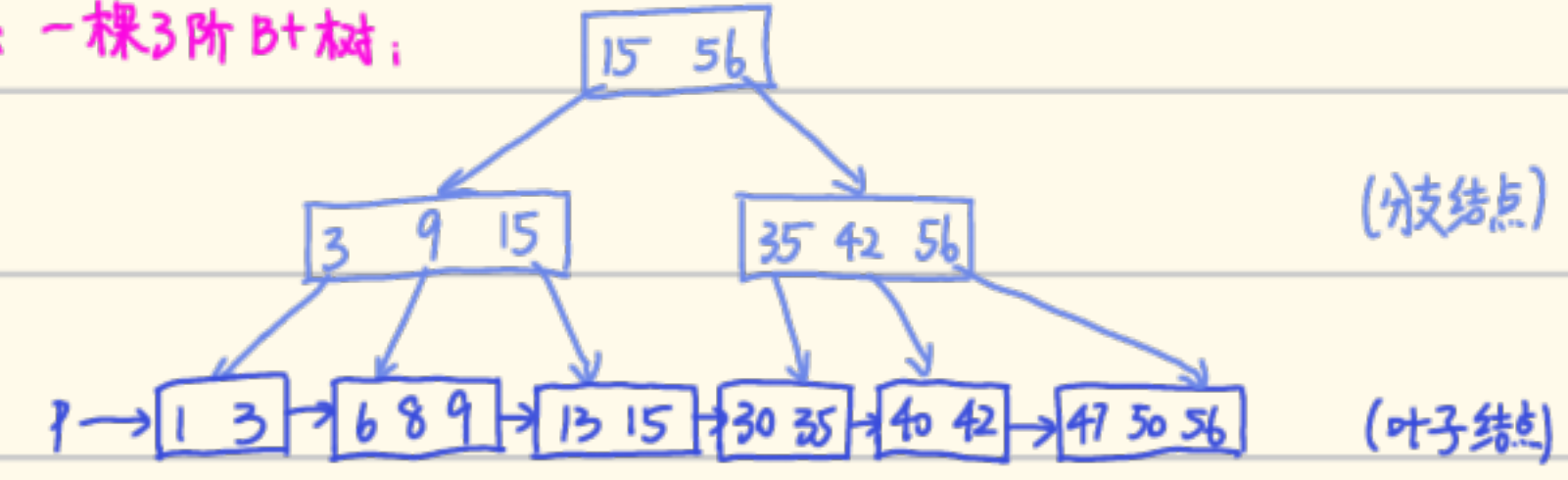
eg: 综合练习 (B 树的删除)



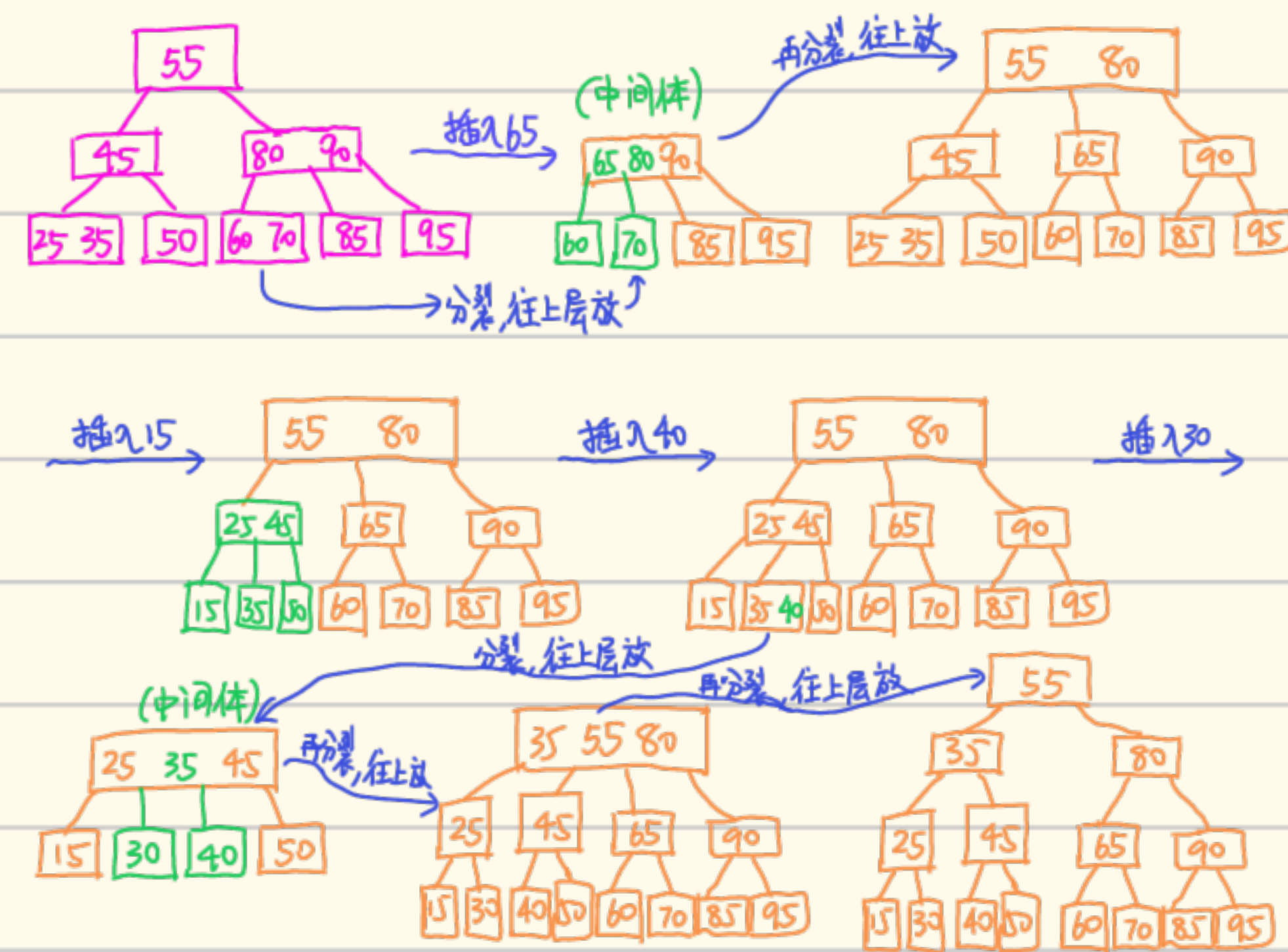
B+ 树:

- 所有关键字都存放于叶结点中，非叶结点存放的是关键字复写以遍查询
- B+ 树的叶结点之间有链接，故可以从最小关键字结点开始顺序搜索。

eg: 一棵 3 阶 B+ 树：



练习：以下 3 阶 B 树，画出分别插入 65, 15, 40, 30 后的变化



练习：以下 3 阶 B 树，画出删除 50, 40 后树的变化

